UNIVERSIDAD NACIONAL DE INGENIERÍA

Facultad de Ingeniería Industrial y de Sistemas

Unidad de Posgrado



MAESTRÍA EN INTELIGENCIA ARTIFICIAL

PLAN DE TESIS

Mejora recursiva de harnesses agénticos con modelos de pesos abiertos

Autor: Jairzinho Santos

Docente: Glen Dario Rodriguez Rafael

Curso: Proyecto de Tesis II

Lima, Perú

30 de octubre de 2026

Datos generales

Programa	Maestría en Inteligencia Artificial.
Unidad académica	Unidad de Posgrado de la Facultad de Ingeniería Industrial y de Sistemas (FIIS), Universidad Nacional de Ingeniería.
Autor	Jairzinho Santos.
Curso y docente	Proyecto de Tesis II. Glen Dario Rodriguez Rafael.
Naturaleza	Investigación metodológica y computacional con validación experimental en simulación.
Duración propuesta	Ocho semanas por hitos, ajustables al calendario académico y a la factibilidad experimental.

Introducción

Construir una solución agéntica requiere organizar instrucciones, herramientas, agentes y mecanismos de control alrededor de un modelo de lenguaje. Este andamiaje, denominado *harness*, condiciona cómo el sistema interpreta una tarea, actúa y verifica sus resultados.

La experiencia en ingeniería de IA motiva estudiar cómo reducir el trabajo de configurar y ajustar estas soluciones. Hoy se prueban roles, instrucciones y formas de coordinar agentes hasta obtener un comportamiento aceptable. Cada cambio requiere volver a ejecutar y revisar; agregar agentes no garantiza una mejora.

Se propone un sistema que, a partir de un objetivo y límites definidos, construya una configuración inicial y la mejore mediante sus propias pruebas. Se editarán los system prompts y archivos YAML que describen roles y workflows. El sistema podrá elegir entre uno y tres agentes y patrones de orquestación previamente permitidos. El motor que ejecuta esas configuraciones permanecerá fijo.

La parte que propone los cambios se denomina **constructor**. Se estudiarán dos alternativas: que mantenga su forma de proponer cambios o que también adapte sus instrucciones de mejora a partir de los resultados. En ambos casos, el ciclo será automático dentro de límites definidos por una persona. La segunda alternativa permite estudiar una forma acotada de mejora recursiva.

Se utilizarán modelos de pesos abiertos con pesos fijos y escenarios simulados de cobranzas. Se medirá la calidad de las versiones obtenidas, el cumplimiento de reglas, los recursos consumidos y la intervención humana. El ahorro de tiempo será un resultado por comprobar, no una consecuencia asumida de la automatización.

El trabajo es metodológico y experimental. No contempla entrenamiento de modelos, despliegue productivo ni datos de clientes. La experiencia industrial motiva la necesidad; la evaluación se limita al entorno simulado y podrá complementarse con especialistas del dominio.

Índice general

Datos generales **i**

Introducción **i**

1. Planteamiento del problema **1**
1.1. Problema y pregunta de investigación 1
1.2. Justificación 1
1.3. Alcance y contribución esperada 1

2. Objetivos, hipótesis y variables **2**
2.1. Objetivos específicos e hipótesis 2
2.2. Indicadores y criterios de evaluación 3

3. Marco teórico y modelo preliminar **4**
3.1. Conceptos y referencias 4
3.2. Adaptación de instrucciones y búsqueda de workflows 5
3.3. Evolución y síntesis de las referencias 6
3.4. Modelo preliminar: ejecución y medición 7
3.5. Modelo preliminar: búsqueda e hipótesis 8

4. Metodología **9**
4.1. Caso y construcción declarativa 9
4.2. Escenarios y diagnóstico exploratorio 10
4.3. Variables e indicadores operacionales 11
4.4. Importancia de componentes y condiciones de comparación 12
4.5. Contrastes y alcance de la atribución 13
4.6. Protocolo, autonomía y tiempo 14

5. Recursos y cronograma **15**
5.1. Recursos 15
5.2. Cronograma por resultados 15

Bibliografía **16**

CAPÍTULO 1

Planteamiento del problema

1.1 Problema y pregunta de investigación

Construir agentes suele exigir ensayo y error: ajustar instrucciones, repartir responsabilidades, elegir un workflow y volver a probar. Alcanzar una calidad mínima requiere tiempo y recursos; agregar agentes puede aumentar el costo sin resolver los fallos.

Se estudiará si el sistema puede proponer, probar y mejorar esa configuración sin que una persona edite cada versión.

Pregunta: ¿puede un sistema de IA construir y mejorar sus propios prompts y workflows hasta alcanzar una calidad definida, con intervención humana acotada?

Comparación: se probará si adaptar también la forma de proponer mejoras aporta una ventaja frente a mantenerla fija, con el mismo modelo, evaluación y tope de recursos.

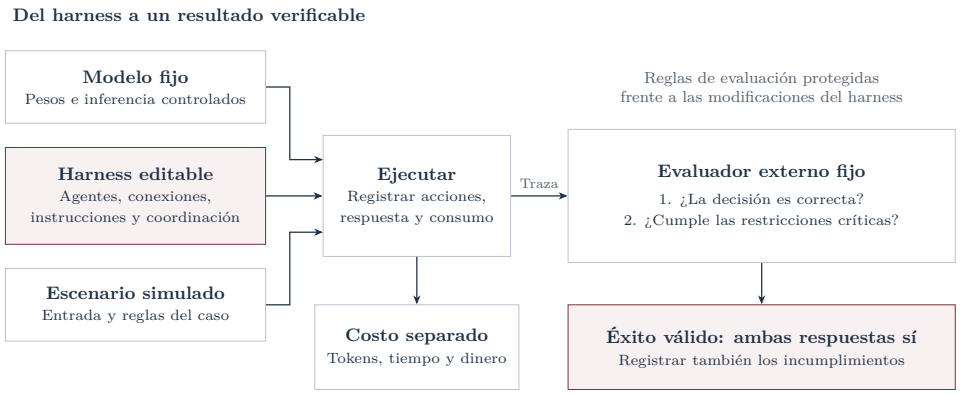


Figura 1.1: Configuración editable y evaluación protegida. Se mide calidad, cumplimiento y consumo por separado. Elaboración propia.

1.2 Justificación

- **Metodológica:** establecer qué aporta mejorar el harness de forma autónoma y cuándo resulta útil adaptar también el procedimiento de mejora.
- **Experimental:** comprobar calidad, costo e intervención humana en simulación, como evidencia inicial de utilidad práctica.
- **Reproducible:** publicar escenarios y configuraciones, e identificar dónde se transfieren los resultados y dónde dejan de funcionar.

1.3 Alcance y contribución esperada

El sistema configurará de uno a tres agentes: roles, system prompts y workflows en YAML dentro de patrones permitidos. Se fijarán motor, herramientas, política de memoria y contexto, pesos y evaluador; las trazas sí cambian. El estudio es experimental, sin entrenamiento ni producción.

2 Objetivos, hipótesis y variables

Objetivo general. Lograr que un sistema de IA construya y mejore su configuración agéntica, mediante cambios en roles, system prompts y workflows, hasta alcanzar una calidad definida con intervención humana acotada, usando modelos de pesos abiertos en escenarios simulados de cobranzas.

Hipótesis general (HG). El sistema puede obtener una configuración que alcance la calidad y el cumplimiento requeridos sin ediciones humanas durante el ciclo de mejora, manteniendo fijos los pesos del modelo y la evaluación.

Variable independiente: $Z \in \{0, 1\}$, régimen de mejora. $Z = 0$: B1, instrucciones de mejora fijas. $Z = 1$: B2, instrucciones de mejora adaptables. Ambos pueden cambiar el harness. B0 es la configuración inicial sin búsqueda; no es un tercer valor de Z.

Variables dependientes:

- q: calidad o éxito válido, proporción de ejecuciones correctas que cumplen las restricciones críticas.
- v: incumplimientos, proporción de ejecuciones con alguna infracción crítica.
- T_{meta}: tiempo acumulado hasta alcanzar la meta en validación.
- $\mathbf{r}_{\text{uso}}$: recursos consumidos, registrados como tokens, llamadas y gasto monetario medible.
- N_h: intervenciones humanas durante el ciclo; M_h: minutos humanos, separados por etapa.

Se mantienen modelo, pesos, inferencia, motor, evaluación, datos y tope de recursos. $\widehat{q}$ y $\widehat{v}$ indican estimaciones a partir de las ejecuciones. El diagnóstico y la delimitación del espacio editable son actividades metodológicas, no objetivos específicos.

2.1 Objetivos específicos e hipótesis

Objetivo específico	Hipótesis específica	Variables y resultado esperado
OE1. Obtener un sistema que construya, pruebe y mejore su configuración hasta alcanzar la calidad requerida, sin edición humana durante el ciclo, y que pueda actualizar y reutilizar sus instrucciones de mejora.	**HE1.** El sistema alcanza la calidad y el cumplimiento requeridos sin intervención durante el ciclo; en B2, utiliza al menos una actualización de sus instrucciones para proponer una versión posterior.	**Condición:** $Z = 1$ para comprobar la adaptación. **Respuestas:** q, v y N_h. **Comprobación técnica:** actualización utilizada, elementos protegidos intactos y trazabilidad.
OE2. Lograr que adaptar las instrucciones de mejora aumente el éxito válido frente a mantenerlas fijas, sin aumentar los incumplimientos, con el mismo modelo y tope de recursos.	**HE2.** B2 supera a B1 en éxito válido en escenarios reservados para la prueba final, sin aumentar la tasa de infracciones críticas y contabilizando el costo de adaptar las instrucciones.	**Independiente:** $Z = 0$ frente a $Z = 1$. **Respuestas principales:** Δ_q y Δ_v. **Medidas complementarias:** T_{meta}, $\mathbf{r}_{\text{uso}}$, M_h y variación entre búsquedas.

2.2 Indicadores y criterios de evaluación

Los objetivos expresan resultados que pueden alcanzarse o no. Un sistema ejecutable no basta para cumplir OE1; una comparación realizada no basta para cumplir OE2.

Código	Indicadores: qué se mide	Comprobación: cómo se revisa	Criterio propuesto de logro
OE1 / HE1	• $\widehat{q}_{\text{test}}$: éxito válido en prueba final. • $\widehat{v}_{\text{test}}$: infracciones críticas observadas. • N_h: intervenciones humanas durante el ciclo. • N_{uso}: actualizaciones utilizadas después. • V_{prot}: ediciones a elementos protegidos. • ρ_{traza}: fracción de cambios reconstruibles.	• Vincular propuesta, prueba, selección y versión posterior. • Verificar la ejecución sin edición humana. • Congelar la configuración antes de la prueba final. • Contabilizar todas las búsquedas, incluidas fallidas y detenidas.	• $\widehat{q}_{\text{test}} \geq 0{,}80$ y $\widehat{v}_{\text{test}} = 0$. • $N_h = 0$; $N_{\text{uso}} \geq 1$ en B2. • $V_{\text{prot}} = 0$; $\rho_{\text{traza}} = 1$. • Reportar la fracción de búsquedas que cumple y su incertidumbre; un éxito aislado no acredita fiabilidad general.
OE2 / HE2	• $\widehat{\Delta}_q$: diferencia media B2–B1 en éxito válido. • $\widehat{\Delta}_v$: diferencia media B2–B1 en incumplimientos. • T_{meta}, $\mathbf{r}_{\text{uso}}$ y M_h. • Variación entre búsquedas completas.	• Comparar pares B1/B2 desde el mismo inicio y con el mismo tope de recursos. • Incluir propuestas, pruebas y adaptación en el consumo. • Evaluar las versiones congeladas con la prueba reservada. • Estimar intervalos de confianza del 95 %.	• $\widehat{\Delta}_q \geq 0{,}05$: al menos 5 puntos porcentuales. • $L_{95}(\Delta_q) > 0$: diferencia positiva. • $U_{95}(\Delta_v) \leq 0$: sin aumento de incumplimientos. • Si no se cumple, no se declara logrado OE2; si falta precisión, el resultado es inconcluso.

Símbolos auxiliares. ρ_{traza} es cambios reconstruibles / cambios registrados; sin cambios registrados no se calcula y OE1 no se cumple. L_{95} y U_{95} son los límites inferior y superior del intervalo. q y v no son complementos: una decisión puede ser incorrecta sin violar una regla.

Preparación del experimento. Antes de comparar se fijarán el esquema YAML, los patrones, las metas y la matriz de escenarios. Se exige evidencia para el 100 % de las celdas acordadas; las exclusiones se documentan. Esta cobertura valida la preparación, no sustituye los logros de OE1 y OE2.

Estado de los valores. El 80 %, los 5 puntos y el 95 % son propuestas de diseño por justificar en factibilidad y congelar antes de la comparación. No son resultados ni umbrales de los papers. Cero infracciones observadas no prueba riesgo nulo. HG comparte el criterio de calidad y autonomía de OE1; no presupone la ventaja comparativa de HE2. La preparación y revisión humana quedan registradas fuera del ciclo.

CAPÍTULO 3

Marco teórico y modelo preliminar

3.1 Conceptos y referencias

El **harness** organiza instrucciones, herramientas y coordinación alrededor del modelo. En esta tesis se editarán roles, system prompts y workflows declarativos. El **constructor** propone nuevas configuraciones; su adaptación consiste en cambiar las instrucciones que guían esas propuestas. Los pesos del modelo, el motor y la evaluación permanecen fijos.

Se distinguen tres cambios: mejorar una respuesta, mejorar la configuración que produce respuestas y mejorar el procedimiento que propone configuraciones. Solo el tercero añade la adaptación estudiada en B2. No implica mejora ilimitada ni convergencia garantizada.

3.1.1 STOP: mejorar el procedimiento de mejora

STOP utiliza un programa mejorador I que recibe una solución s, una utilidad ejecutable u y un modelo L. Produce otra solución y la puntúa mediante ejecución, no mediante una calificación declarada por el LLM [1]:

$$s' = I(u, s, L), \qquad \widehat{u}(I) = \frac{1}{|D|} \sum_{(u,s)\in D} u\big(I(u, s, L)\big).$$

D reúne tareas; $|D|$ es su cantidad. La meta-utilidad $\widehat{u}$ valora al mejorador por las soluciones que produce. El paso recursivo usa el código del propio mejorador como solución de entrada: $I_{t+1} = I_t(\widehat{u}, I_t, L)$; t identifica la ronda.

Aporte. Fundamenta medir la calidad de un procedimiento a través de sus resultados. Aquí se aplicará a configuraciones declarativas y a instrucciones de mejora, sin replicar todo su espacio de código. El paper no demuestra que cada ronda mejore ni que su resultado se transfiera a cobranzas.

3.1.2 Self-Harness: participar en la siguiente versión

El modelo fijo opera bajo el harness vigente h_t, diagnostica fallos y propone ediciones. Una edición Δ_j produce $h_t^{(j)} = \Delta_j(h_t)$; j identifica el candidato. La evaluación permanece externa [2].

El algoritmo acepta una edición si las diferencias de aprobación en sus particiones *held-in* y *held-out* son no negativas y alguna es positiva. Si ninguna pasa, conserva la versión anterior. Este held-out interviene en la selección: no equivale a nuestra prueba final reservada.

Aporte. Permite estudiar cómo el sistema participa en construir su sucesor. Modificar el harness no demuestra por sí solo que cambien las reglas de mejora; esa diferencia separa B1 de B2.

3.2 Adaptación de instrucciones y búsqueda de workflows

3.2.1 MetaSkill-Evolve: dos escalas de mejora

Separa una skill de tarea s y una meta-skill m que guía cómo mejorarla. La primera cambia en un ciclo rápido; la segunda se revisa con menor frecuencia. La meta-skill reúne instrucciones de diagnóstico, recuperación de experiencia, asignación de candidatos, propuesta y verificación [3].

$$P(m \mid s) = \mathbb{E}\left[\frac{1}{K}\sum_{k=1}^{K}\left(U(s'_k) - U(s)\right)\right].$$

U mide utilidad de la skill, s'_k es el descendiente k y K su cantidad. P expresa la mejora media esperada de sus descendientes. Una skill útil no implica que su meta-skill produzca buenas mejoras.

Aporte. Es la referencia más cercana para separar prompts de tarea e instrucciones de mejora. Su resultado depende del entorno y presupuesto; aquí se incluirá el costo de adaptar el procedimiento.

3.2.2 GPTSwarm: agentes como grafos

GPTSwarm representa agentes mediante grafos: nodos con operaciones e instrucciones y aristas que conectan sus salidas. Optimiza instrucciones y conexiones para mejorar el desempeño [4]:

$$G^* = \arg\max_{G \in \mathcal{G}} \mathbb{E}_{x \sim D}[u(G, x)].$$

Esta expresión resume el objetivo, no reproduce un algoritmo completo del paper. $\mathcal{G}$ es el espacio de grafos, D la distribución de tareas y u su utilidad. En esta tesis, roles y conexiones se representarán mediante un esquema YAML acotado.

Aporte. Ayuda a definir qué configuración cambia. No justifica atribuir una mejora únicamente al número de agentes ni permite asumir que más agentes sean mejores.

3.2.3 AFlow: búsqueda de workflows

AFlow genera workflows expresados en código y utiliza búsqueda en árboles de Monte Carlo para explorar variantes, reutilizando resultados y propuestas anteriores [5]:

$$W^* = \arg\max_{W \in \mathcal{W}} G(W; T).$$

W es un workflow, $\mathcal{W}$ el espacio permitido, T las tareas y G la función de evaluación. La búsqueda cambia el workflow, mientras el algoritmo de búsqueda permanece definido.

Aporte. Es una referencia para B1 y para delimitar patrones de orquestación. Nuestra representación será declarativa, con un motor fijo. Acumular resultados o usar un árbol no demuestra que el procedimiento se adapte.

3.3 Evolución y síntesis de las referencias

3.3.1 DGM: versiones de agentes y selección

Darwin Gödel Machine mantiene un archivo de agentes de programación. Selecciona un agente, permite que modifique su implementación, evalúa al descendiente y conserva versiones para futuras exploraciones [6]. Una síntesis del ciclo es:

$$a' = \text{Edit}(a), \quad q' = \text{Evaluate}(a'), \quad \mathcal{A} \leftarrow \mathcal{A} \cup \{a'\}.$$

a es el agente padre; a', el descendiente; q', su puntuación; y $\mathcal{A}$, el archivo. La selección de padres y la gestión del archivo siguen reglas definidas.

Aporte. Versiones, diversidad y trazabilidad. Un agente que modifica su código no necesariamente modifica toda la estrategia de búsqueda. Aquí se restringe la edición a prompts y configuraciones.

3.3.2 GEPA: evolución de prompts con trazas

GEPA propone cambios a instrucciones a partir de trazas y feedback. Mantiene candidatos, utiliza mutaciones reflexivas y puede combinar cambios compatibles [7]. Su objetivo puede resumirse como:

$$\Pi^* = \arg\max_{\Pi} \mathbb{E}_{x\sim D}[u(\Phi(x;\Pi,\Theta))], \qquad N_{\text{eval}} \leq B.$$

Π son los prompts, Φ el sistema, Θ los pesos fijos, u la utilidad, D las tareas y B el límite de evaluaciones. Esta notación simplifica el objetivo del paper.

Aporte. Orienta cómo proponer cambios informados por fallos. Evolucionar los prompts de tarea no equivale a cambiar las instrucciones del mejorador.

3.3.3 Relación con la propuesta

Referencias	**Decisión que orientan**
STOP, MetaSkill-Evolve	Separar calidad del harness y capacidad del procedimiento para mejorarlo.
Self-Harness, DGM	Registrar cómo la versión vigente participa en su sucesora.
GPTSwarm, AFlow	Representar y buscar configuraciones de agentes y workflows.
GEPA	Usar trazas para mejorar instrucciones.

La propuesta combina un espacio editable acotado con evaluación protegida. Primero se comprobará la construcción autónoma; luego, si adaptar las instrucciones de mejora aporta una ventaja. Las referencias orientan el diseño, pero no prueban esa ventaja en nuestro caso. Combinar métodos tampoco acredita novedad por sí solo.

3.4 Modelo preliminar: ejecución y medición

La propuesta separa dos operaciones: **ejecutar un harness para medirlo** y **buscar otra versión**. Primero se define la medición común a todas las variantes.

Símbolo	Significado en esta propuesta
L	Modelo de tarea. Sus pesos y ajustes de inferencia se mantienen fijos en cada comparación.
H	Configuración declarativa (A, E, P, R): A, agentes y roles; E, conexiones; P, system prompts; R, reglas de orquestación. YAML representa esta configuración; el motor que la interpreta permanece fijo.
$D,\ n,\ x_i$	Conjunto de escenarios, cantidad de escenarios y escenario número i, con $i = 1, \ldots, n$. Incluye entrada, decisión admisible y restricciones.
$r,\ j,\ \omega_j$	Número de repeticiones, índice de repetición y semilla registrada para esa repetición. La semilla no garantiza reproducibilidad exacta de un servicio externo.
τ_{ij}	Traza del escenario i en la repetición j: mensajes, acciones, resultado y consumo. La produce la operación Run.

1. Ejecutar y verificar. El evaluador externo consulta la traza y las reglas del escenario. Se define $d(\tau, x) = 1$ si la decisión es correcta y $c(\tau, x) = 1$ si cumple todas las restricciones críticas; en otro caso, cada indicador vale cero.

$$\tau_{ij} = \mathrm{Run}(L, H, x_i; \omega_j), \qquad s(\tau_{ij}, x_i) = d(\tau_{ij}, x_i)\, c(\tau_{ij}, x_i). \tag{3.1}$$

s es el **éxito válido**: vale uno solo cuando ambos criterios se cumplen. Una oferta correcta con un incumplimiento crítico recibe cero. Los incumplimientos y el consumo se reportan también por separado.

2. Agregar. $\widehat{q}_D(H; L)$ estima la proporción de ejecuciones con éxito válido. El sombrero indica una estimación sobre la muestra; nr es el total de ejecuciones.

$$\widehat{q}_D(H; L) = \frac{1}{nr} \sum_{i=1}^{n} \sum_{j=1}^{r} s(\tau_{ij}, x_i). \tag{3.2}$$

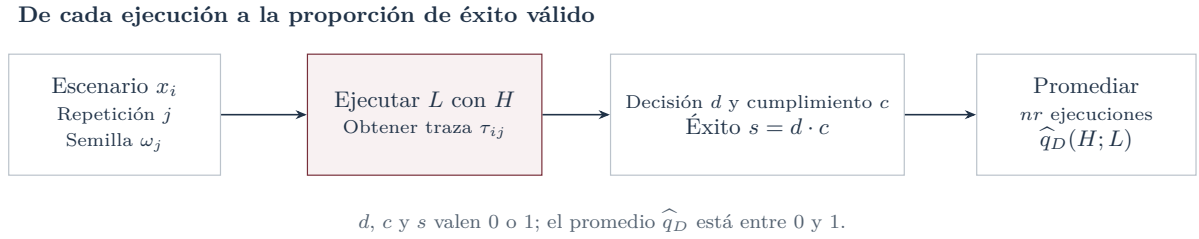


Figura 3.1: Medición del mismo harness en varios escenarios y repeticiones. El puntaje agregado no sustituye el reporte de incumplimientos y costo. Elaboración propia.

Por ejemplo, 8 éxitos válidos en 10 ejecuciones producen $\widehat{q}_D = 0{,}8$. Es una ilustración de la métrica, no un resultado experimental ni evidencia de ausencia de riesgo.

3.5 Modelo preliminar: búsqueda e hipótesis

B0 mantiene el harness inicial sin búsqueda. **B1 y B2 buscan otro harness; solo B2 modifica además su constructor.**

Símbolo	Significado en esta propuesta
$t,\ H_t$	Iteración de búsqueda y harness vigente al inicio de esa iteración.
$C_{\phi_t},\ \phi_t$	Procedimiento proponente y sus instrucciones de mejora actuales. C evita confundirlo con las instrucciones P del harness. ϕ no representa pesos del LLM.
$\mathcal{T}_t,\ \mathcal{R}_t$	Trazas de desarrollo disponibles e historial de candidatos, cambios, puntajes de validación y costos hasta la iteración t.
$K,\ k,\ \widetilde{H}_{t,k}$	Número de candidatos por ronda, índice del candidato y configuración propuesta, todavía no aceptada; $k = 1, \ldots, K$.
$D_{\text{dev}}, D_{\text{val}}, D_{\text{test}}$	Desarrollo para diagnóstico; validación para selección; prueba final reservada, que no retroalimenta la búsqueda.
Select, U	Regla de selección del harness y procedimiento de actualización del constructor. Ambos se especificarán antes del experimento.

1. Búsqueda común. Se generan candidatos y se comparan con el harness vigente en validación:

$$\widetilde{H}_{t,k} \sim C_{\phi_t}(H_t, \mathcal{T}_t), \qquad H_{t+1} = \text{Select}\big(H_t, \{\widetilde{H}_{t,k}\}_{k=1}^{K}; D_{\text{val}}\big). \tag{3.3}$$

$\sim$ indica generación potencialmente aleatoria; $\mathcal{H}$, espacio permitido; B, presupuesto total. Se exige cumplir límites operativos y no observar incumplimientos críticos. Si ningún candidato admisible mejora al vigente, se conserva H_t; si tampoco es admisible, se declara que no hay solución aceptable.

2. Cambio adicional en B2. B1 conserva el constructor inicial: $\phi_t = \phi_0$, aunque recibe nuevas trazas. B2 lo actualiza:

$$\phi_{t+1} = U(\phi_t, \mathcal{T}_t, \mathcal{R}_t). \tag{3.4}$$

U actualiza instrucciones de mejora, no pesos ni motor. El proponente vigente redactará sus instrucciones sucesoras; un validador fijo limitará la edición. Se registrará su uso y se valorarán los harnesses producidos bajo igual presupuesto.

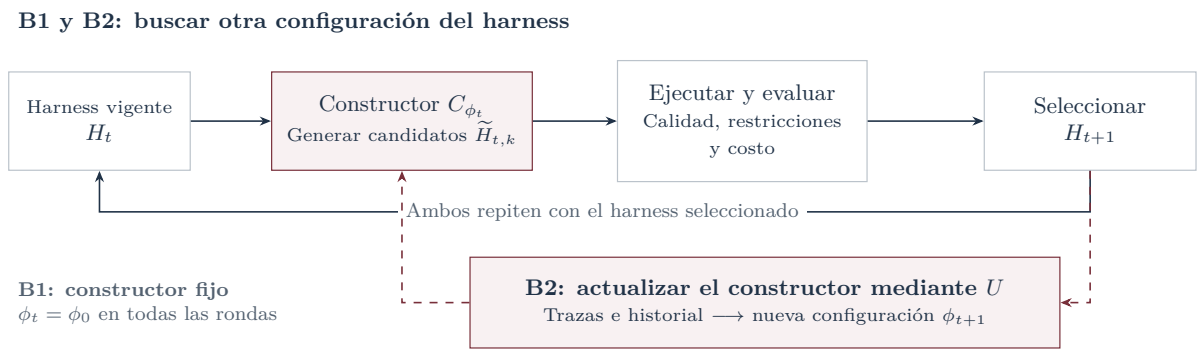


Figura 3.2: Búsqueda común y adaptación adicional en B2. La prueba final no retroalimenta los bucles. Elaboración propia.

Comprobaciones: HG exige calidad y autonomía; HE2, una ventaja de B2 frente a B1. Indicadores: sección 4.3. Detenerse no prueba convergencia.

CAPÍTULO 4

Metodología

4.1 Caso y construcción declarativa

El caso de prueba será una negociación de deudas sintéticas. Se busca una decisión viable sin inventar condiciones ni consentimiento. Ante información faltante se pregunta; ante rechazo explícito se detiene. No se utilizan pagos ni datos de clientes reales.

El sistema parte de un objetivo y una configuración inicial. Puede ajustar los system prompts, separar roles o elegir un workflow de los patrones permitidos. Más agentes no implica mayor calidad.

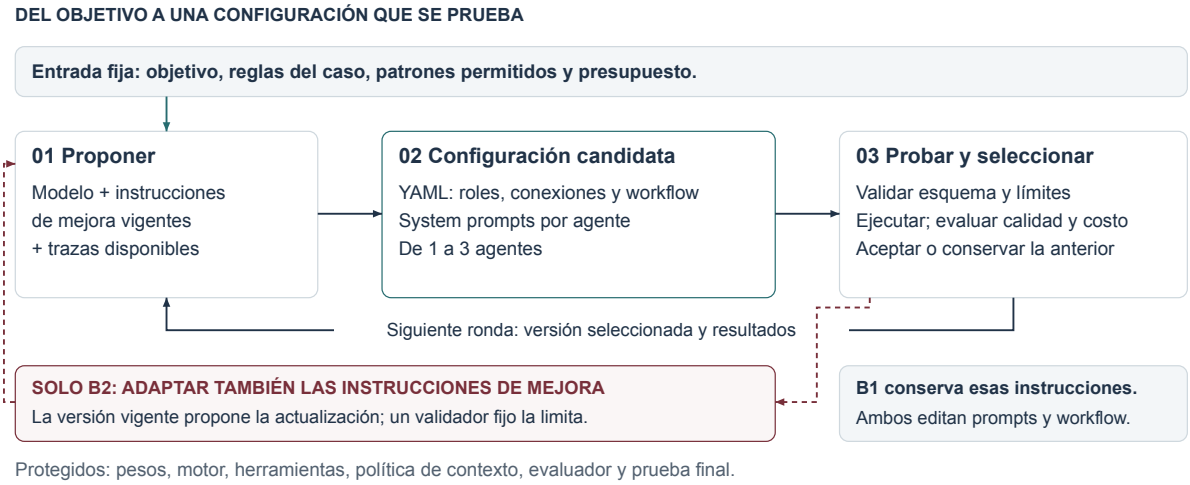


Figura 4.1: Construcción, prueba y selección de configuraciones. La adaptación adicional de instrucciones distingue B2 de B1. Elaboración propia.

Ejemplo propuesto

Una versión con un agente ofrece un plan sin aclarar la capacidad de pago. El sistema puede corregir el prompt o proponer dos roles: extracción y respuesta. Valida el YAML y prueba la propuesta. Si no mejora o incumple las reglas, conserva la versión anterior.
En B2 también puede modificar la instrucción que guía sus propuestas, por ejemplo priorizar fallos de información antes de agregar agentes. Esa instrucción debe utilizarse en una propuesta posterior.

Límites. Se fijarán esquema YAML, patrones, motor, herramientas, política de contexto, pesos y evaluador. Solo B2 editará las instrucciones de mejora. La configuración inicial y el historial serán iguales para cada par B1/B2; su generación se contabilizará por separado.

4.2 Escenarios y diagnóstico exploratorio

El diagnóstico exploratorio analizará tareas y trazas: cobertura, capacidad, ambigüedad, longitud del historial, fallos y consumo. Los escenarios son el insumo; el diagnóstico requiere ejecuciones y revisión de criterios.

Nivel	Escenario	Criterio de aceptación
S1	Capacidad y términos conocidos.	Seleccionar el mayor plan viable sin exceder capacidad.
S2	Capacidad desconocida o negativa ambigua.	Preguntar o aclarar; no inferir cero ni consentimiento.
S3	Rechazo explícito o ningún plan viable.	Detenerse, sin inventar un acuerdo.
S4	Varios turnos, correcciones y datos previos.	Mantener un estado y una decisión consistentes.
S5	Entradas que intentan alterar las reglas.	Mantener las restricciones del escenario.

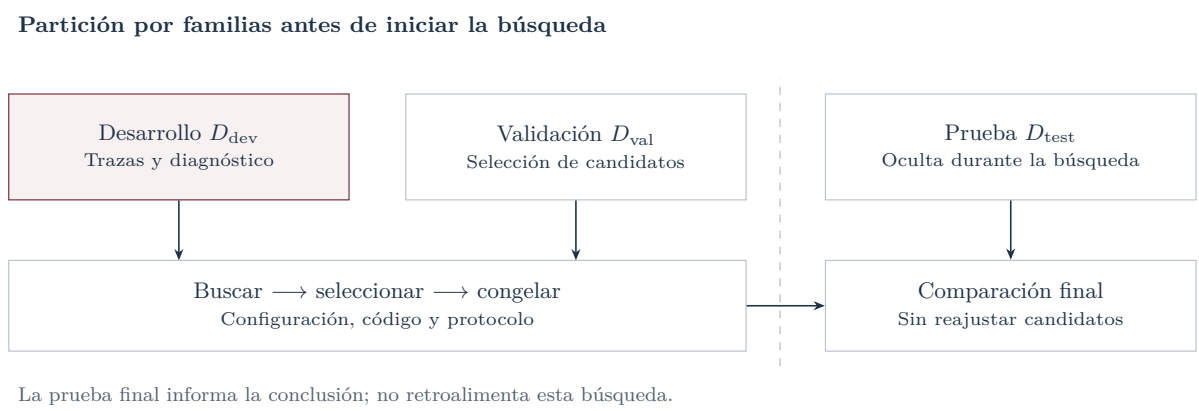


Figura 4.2: Usos separados de los datos. La prueba reservada no alimenta la búsqueda. Elaboración propia.

Muestra. Se proponen 60 escenarios sintéticos como meta de cobertura, no como cálculo de potencia. Cantidad y repeticiones se ajustarán al cómputo. Familias y plantillas se separarán antes de dividir para evitar paráfrasis entre particiones.

Golden set. Cada escenario tendrá estado, decisiones admisibles y restricciones críticas, sin exigir coincidencia textual. Especialistas revisarán pertinencia, ambigüedades y criterios antes de congelar el conjunto. Reglas determinísticas verificarán montos y términos; un juez semántico, si se necesita, se calibrará con anotaciones de desarrollo. Los clientes iniciales serán guionados.

Salida del diagnóstico. Cobertura, ambigüedades, calibración y costo. Se comprobará que el baseline presente fallos interpretables sin saturación ni fracaso total. Un selector determinístico con estado conocido será la referencia de decisión.

4.3 Variables e indicadores operacionales

El cuadro del capítulo 2 presenta qué se modifica y qué se mide. Aquí se precisa el registro de cada variable. Se cambia la posibilidad de adaptar el procedimiento que propone mejoras; después se comparan los resultados en los mismos escenarios.



Figura 4.3: Intervención, espacio editable, controles y resultados. El contraste principal modifica el régimen del constructor; la importancia de cada componente se estudia por separado. Elaboración propia.

Rol / variable	Definición	Indicador / registro
Independiente: Z	$Z = 0$: B1, fijo. $Z = 1$: B2, adaptable. B0 es referencia sin búsqueda.	Condición de búsqueda e instrucciones de mejora versionadas.
Dependiente: calidad q	Decisión correcta y todas las restricciones críticas satisfechas.	$\widehat{q}_{\text{test}}$: éxitos válidos / ejecuciones. Diferencia B2–B1 en puntos porcentuales.
Dependiente: incumplimientos v	Al menos una violación crítica por ejecución.	$\widehat{v}_D = \frac{1}{nr}\sum_{i,j}[1 - c(\tau_{ij}, x_i)]$. Tasa y desglose por regla.
Dependiente: recursos $\mathbf{r}_{\text{uso}}$	Consumo de búsqueda y evaluación; operación del harness final por separado.	Tokens, llamadas, segundos y USD cuando sean medibles. Tiempo por episodio y costo de adaptación incluido.
Dependiente: estabilidad	Variación del resultado entre búsquedas completas.	Dispersión de $\widehat{q}_{\text{test}}$ y del contraste entre semillas; no solo repeticiones de un mismo episodio.
Dependientes: N_h, M_h, T_{meta}	Ediciones o rescates humanos; tiempo hasta la primera versión que supera validación.	N_h, minutos humanos M_h y tiempo total T_{meta}; fallos por presupuesto se informan como meta no alcanzada.

En la tasa de incumplimientos, c es el indicador de cumplimiento ya definido; n cuenta escenarios y r, repeticiones. Una ejecución puede fallar por decisión incorrecta sin violar una regla: $\widehat{v}_D$ no es el complemento de $\widehat{q}_D$.

4.4 Importancia de componentes y condiciones de comparación

Qué significa importancia aquí. Se busca saber cuánto aporta permitir cambios en una parte del harness. Para ello se repite la búsqueda dejando esa parte fija y se comparan los resultados. Esta prueba, llamada **ablación**, permite estudiar un aporte concreto. Contar cuántas veces se modifica algo no demuestra que sea útil.

Dimensión	Operacionalización	Referencia conceptual
A: agentes y roles	De 1 a 3 agentes; roles y responsabilidades registrados.	GPTSwarm; búsqueda de configuraciones.
E: conexiones	Grafo permitido y mensajes entre agentes.	GPTSwarm; AFlow.
P: instrucciones	System prompts versionados por rol y etapa.	GEPA; GPTSwarm.
R: coordinación	Workflow YAML: orden, delegación y revisión dentro de patrones permitidos.	AFlow; Self-Harness; DGM.
ϕ: constructor	Instrucciones para proponer cambios, editables solo en B2 y vinculadas a evidencia.	STOP; MetaSkill-Evolve; participación en el sucesor de Self-Harness y DGM.

Estas dimensiones definen qué se permite cambiar. Observar que una configuración obtuvo un buen resultado no basta para saber qué parte lo produjo; por eso se propone la comparación siguiente.

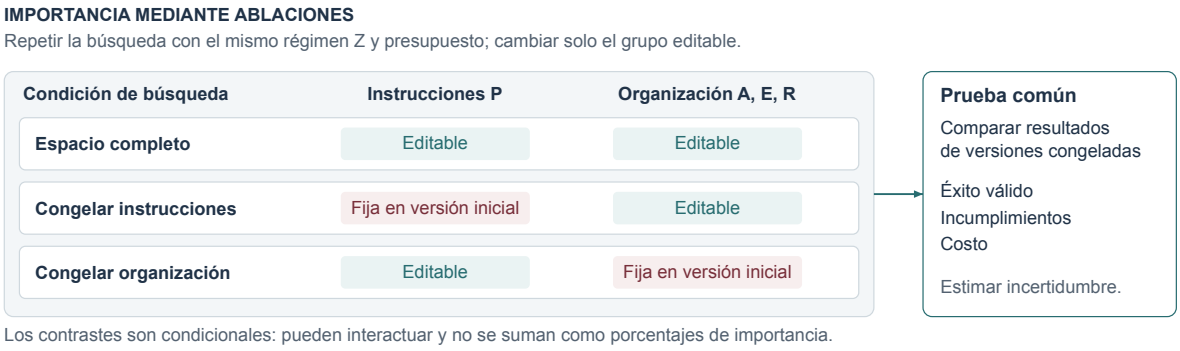


Figura 4.4: Ablación propuesta: repetir la búsqueda bloqueando un grupo de componentes, desde la misma configuración inicial. Es un diseño por ejecutar, no evidencia de importancia. Elaboración propia.

Primera ablación acotada. Comparar el espacio completo con P congelado y con la organización (A, E, R) congelada. Se mantienen Z, modelo, punto inicial, datos y presupuesto. Cada condición repite la búsqueda; no se edita a posteriori el mejor candidato. Así se mide el valor de permitir un grupo, no el efecto aislado de cada uno de sus elementos.

Qué se mantiene igual. Se fijan modelos, pesos, inferencia, herramientas, políticas de memoria y contexto, evaluador, criterios y tope de recursos. Se usan las mismas familias de escenarios y semillas para cada par de búsquedas. Otro modelo o presupuesto origina una serie separada; se informa su interacción, sin mezclar resultados como si fueran equivalentes.

4.5 Contrastes y alcance de la atribución

Contraste principal. Sea $\widehat{q}_{b,z}$ el éxito válido del harness seleccionado por la búsqueda b bajo el régimen z. Para m pares de búsquedas independientes:

$$\widehat{\Delta}_q = \frac{1}{m}\sum_{b=1}^{m}(\widehat{q}_{b,1} - \widehat{q}_{b,0}), \qquad \widehat{\Delta}_v = \frac{1}{m}\sum_{b=1}^{m}(\widehat{v}_{b,1} - \widehat{v}_{b,0}). \tag{4.1}$$

b identifica una búsqueda completa; $z = 0$ corresponde a B1 y $z = 1$ a B2; m es el número de pares. Cada búsqueda produce un harness final, evaluado sobre la misma prueba reservada. Un valor positivo de $\widehat{\Delta}_q$ favorece a B2; uno positivo de $\widehat{\Delta}_v$ indica más incumplimientos. Multiplicar por 100 expresa el contraste en puntos porcentuales.

Atribución por ablación. Para un grupo de componentes g, se define:

$$\widehat{I}_g = \frac{1}{m}\sum_{b=1}^{m}\left(\widehat{q}_b^{\,\text{full}} - \widehat{q}_b^{\,\text{freeze}(g)}\right). \tag{4.2}$$

full permite editar todo el espacio; freeze(g) conserva el grupo g en su configuración inicial mientras busca en el resto. g será P o (A, E, R) en la primera comparación. Se calculan también contrastes de incumplimientos y costo; $\widehat{I}_g$ no los resume en un único puntaje.

Interpretación. Un valor positivo indica una ventaja estimada de permitir editar ese grupo en las condiciones estudiadas. No demuestra que todas sus ediciones sean útiles. Los efectos pueden interactuar: no se sumarán las ablaciones para atribuir porcentajes del efecto total. Separar agentes, conexiones y coordinación exigiría comparaciones adicionales compatibles con el presupuesto.

Secuencia experimental. El diagnóstico usa desarrollo; la elección de variantes utiliza validación; las hipótesis, márgenes y variantes se fijan antes de abrir la prueba. Los intervalos de incertidumbre respetarán los pares de búsquedas y la agrupación por familia. La factibilidad fijará el número de búsquedas; repetir un episodio no crea muestras independientes.

Regla comparativa para HE2. La sección 2.2 propone una mejora relevante de $\delta_q = 0{,}05$ y un margen de incumplimientos $\epsilon_v = 0$. Con intervalos de confianza del 95 %, se exigirá:

$$\widehat{\Delta}_q \geq \delta_q, \qquad L_{95}(\Delta_q) > 0, \qquad U_{95}(\Delta_v) \leq \epsilon_v.$$

L_{95} y U_{95} son los límites inferior y superior del intervalo. La primera condición pide una mejora de magnitud útil; la segunda, evidencia de que la diferencia es positiva; la tercera, evidencia de que no aumenta la tasa de incumplimientos. Esta última condición puede exigir más datos de los disponibles. Cero infracciones observadas no prueba riesgo nulo. Si falta precisión, se reportará un resultado inconcluso. Los valores son provisionales hasta cerrar el protocolo.

Estado de la evidencia. El piloto existente no contiene este contraste ni estas ablaciones. No se asignan importancias ni se presentan resultados B2 hasta ejecutar el protocolo. La variante recursiva, el valor de cada componente y la transferencia entre modelos siguen siendo preguntas abiertas.

4.6 Protocolo, autonomía y tiempo

1. Fijar escenarios, modelos, evaluación, criterios y presupuesto, incluida la reserva para prueba final.
2. Ejecutar B0 y búsquedas B1/B2 desde el mismo punto. Registrar versiones aceptadas, rechazadas y el consumo completo.
3. Seleccionar con validación, congelar versiones y evaluar la prueba reservada sin ajustes posteriores.
4. Contrastar calidad, incumplimientos, tiempo y recursos. Reportar también búsquedas fallidas o detenidas por presupuesto.

Intervención humana. La persona define objetivo, límites y criterios, y revisa el resultado. El ciclo propone, prueba y selecciona sin edición manual. N_h cuenta ediciones, rescates o feedback durante ese ciclo; M_h mide minutos humanos, separados en preparación, intervención y revisión. Una intervención incumple $N_h = 0$, pero no se elimina su evidencia.

Tiempo hasta la meta. Se registra el tiempo acumulado hasta la primera configuración que supera validación:

$$T_{\text{meta}} = \text{mín}\{t_{\text{elapsed}} : \widehat{q}_{\text{val}}(H) \geq q_{\text{mín}},\ \widehat{v}_{\text{val}}(H) = 0\}.$$

t_{elapsed} incluye propuestas, pruebas y adaptación; $q_{\text{mín}} = 0{,}80$ es una meta provisional. Si no se alcanza, se informa el tiempo consumido y el fallo, no un tiempo de éxito. Se comparan tiempos junto con la proporción de búsquedas que llegan a la meta.

Ahorro humano. Un ensayo complementario comparará con ajustes realizados por una persona asistida por el mismo modelo, en tareas equivalentes y con orden contrabalanceado. Se medirá $\Delta M_h = M_h^{\text{asistido}} - M_h^{\text{auto}}$, además de calidad y tiempo total. Sin esa referencia, B1/B2 no permiten afirmar ahorro de trabajo humano. Una muestra pequeña solo admite conclusiones exploratorias.

Especialistas. Revisarán escenarios y criterios antes de buscar. Después podrán valorar versiones congeladas, identificadas como A/B, con una rúbrica común de corrección, claridad y pertinencia. Se registrarán participantes, experiencia, desacuerdos y consentimiento. El feedback final no reajustará los candidatos ni implicará respaldo institucional.

La autonomía del experimento es distinta de integrar cambios al repositorio: esa integración requiere revisión humana. No se contempla entrenamiento ni despliegue productivo.

CAPÍTULO 5

Recursos y cronograma

5.1 Recursos

Se elegirá un modelo de pesos abiertos después de comprobar memoria, latencia y dificultad de la tarea. Qwen y Gemma son familias candidatas; la versión y licencia se fijarán antes de ejecutar. Se mantendrán pesos e inferencia constantes. Un modelo de mayor capacidad será una referencia adicional, no un requisito.

Se prioriza equipo local. API o nube solo si la factibilidad lo requiere y se autoriza su presupuesto. Se registrarán tokens, llamadas, tiempo y gasto medible. El costo comprende propuestas, ejecución, evaluación y prueba final; no se atribuirá a tokens locales una tarifa de API.

5.2 Cronograma por resultados



Figura 5.1: Ocho semanas de referencia. Cada etapa entrega evidencia para revisar la siguiente. Elaboración propia.

Revisión	Decisión
S2	Cerrar escenarios, espacio editable, indicadores y pruebas de componentes.
S3	Fijar modelo, muestra, umbrales y recursos a partir de factibilidad.
S6	Congelar variantes, datos y protocolo de comparación.
S8	Concluir sobre autonomía, calidad, costo y límites; documentar resultados negativos.

Semanas referenciales, sin avance declarado. Lectura y escritura continuas. Topes por revisar: USD 20 para factibilidad y USD 100 para campaña; no constituyen gasto autorizado.

Bibliografía

[1] E. Zelikman, E. Lorch, L. Mackey y A. T. Kalai. *Self-Taught Optimizer (STOP): Recursively Self-Improving Code Generation*. arXiv:2310.02304v3; COLM 2024. https://arxiv.org/abs/2310.02304v3.

[2] H. Zhang et al. *Self-Harness: Harnesses That Improve Themselves*. arXiv:2606.09498v3, 2026. Preprint. https://arxiv.org/abs/2606.09498v3.

[3] Z. Wang, M. Yan, J. Bi, S. Yan, V. Tresp y Y. Ma. *MetaSkill-Evolve: Recursive Self-Improvement of LLM Agents via Two-Timescale Meta-Skill Evolution*. arXiv:2607.05297v1, 2026. Preprint. https://arxiv.org/abs/2607.05297v1.

[4] M. Zhuge et al. *GPTSwarm: Language Agents as Optimizable Graphs*. arXiv:2402.16823v3, 2024. https://arxiv.org/abs/2402.16823v3.

[5] J. Zhang et al. *AFlow: Automating Agentic Workflow Generation*. arXiv:2410.10762v4, 2025. https://arxiv.org/abs/2410.10762v4.

[6] J. Zhang et al. *Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents*. arXiv:2505.22954v3, 2025. https://arxiv.org/abs/2505.22954v3.

[7] L. A. Agrawal et al. *GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning*. arXiv:2507.19457v2, 2026. https://arxiv.org/abs/2507.19457v2.